

SYNTHESE ACTIONNABLE

Architecture des Agents IA

Agent Developpeur & Agent SLIDES
Orchestration, RAG, Memoire & Securite

Extraction et synthese des principes fondamentaux, patterns d'orchestration, architectures de reference et strategies de production issus du Compilateur d'Architecture Intelligente, appliques a la conception de deux agents IA de classe mondiale.

Table des Matieres

1. Introduction : Pourquoi cette synthese ?	4
2. Principes Meta-Cognitifs appliques aux Agents IA	4
2.1 Raisonnement par Premiers Principes	4
2.2 Theorie des Contraintes (TOC)	4
2.3 Compression Architecturale	5
2.4 Reversibilite Decisionnelle	5
2.5 Pensee Systemique - Les Leviers	5
3. Patterns d'Orchestration Multi-Agents	5
3.1 Agent Developpeur : Architecture Pipeline + Debat	6
3.2 Agent SLIDES : Architecture Supervisor	6
4. Architecture RAG de Production	7
4.1 Pipeline d'Ingestion	7
4.2 Pipeline de Requete	7
5. Routeur de Modeles Intelligent	8
6. Hierarchie de Memoire Persistante	8
6.1 Techniques de Compression de Memoire	9
7. Boucle d'Auto-Amelioration	9
8. Securite IA en Profondeur	10

9. Architecture Cible : Agent Developpeur	11
9.1 Vue d'Ensemble	11
9.2 Infrastructure Partagee	11
10. Architecture Cible : Agent SLIDES	12
10.1 Vue d'Ensemble	12
10.2 Flux de Creation d'une Presentation	12
10.3 Generation de Visuels	13
11. Stack Technologique Recommandee	13
11.1 Stack AI-Native (Levier 10x)	14
12. Anti-Patterns a Eviter	14
12.1 Microservices Prematures pour Agents	14
12.2 Cache-First Aveugle	14
12.3 Observability Absente	14
12.4 Isolation Multi-Tenant Cassee	15
12.5 Sur-Ingenierie	15
13. Roadmap d'Implementation (12 Semaines)	15
13.1 Phase Fondation (Semaines 1-3)	15
13.2 Phase Intelligence (Semaines 4-6)	16
13.3 Phase Memoire (Semaines 7-8)	16
13.4 Phase SLIDES (Semaines 9-10)	16
13.5 Phase Production (Semaines 11-12)	16

1. Introduction : Pourquoi cette synthese ?

Le document original, intitule "Compilateur d'Architecture Intelligente", constitue une reference encyclopedique couvrant l'ensemble de l'ingenierie logicielle, de la theorie des files d'attente aux architectures hyperscale en passant par le FinOps et la securite DevSecOps. Cependant, pour construire effectivement un Agent Developpeur et un Agent SLIDES, une grande partie de ce contenu n'est pas directement applicable. Ce document extrait, filtre et restructure les connaissances essentielles du document original pour les mapper specifiquement aux besoins de la conception et de l'implementation de ces deux agents IA.

L'objectif n'est pas de reproduire l'integralite du document source, mais d'en distiller les principes fondamentaux, les patterns d'orchestration, les architectures de reference et les strategies de mise en production qui sont directement actionnables pour batir des agents IA de classe mondiale. Chaque section identifie les elements du document original qui sont critiques, ceux qui sont utiles mais secondaires, et ceux qui peuvent etre differees.

Les deux agents cibles partagent une infrastructure commune (routeur de modeles, pipeline RAG, memoire persistante, securite en profondeur) mais divergent dans leur orchestration specifique : l'Agent Developpeur necessite un pipeline sequentiel avec validation multi-couches, tandis que l'Agent SLIDES requiert un orchestrateur supervisor avec des sous-agents specialises en design, contenu et mise en page. Cette synthese articule les deux architectures en mettant en evidence les composants partageables et les specialisations necessaires.

2. Principes Meta-Cognitifs appliques aux Agents IA

Le document original identifie sept modeles mentaux d'architectes d'elite. Voici comment chacun s'applique specifiquement a la conception d'agents IA, avec des recommandations actionnables.

2.1 Raisonnement par Premiers Principes

Ce modele est le plus fondamental pour la conception d'agents. Plutot que de copier l'architecture d'un agent existant (comme Genspark ou Cursor), il faut decomposer les besoins jusqu'aux axiomes invariants. Pour un Agent Developpeur, les axiomes sont : (1) le code doit etre syntaxiquement correct, (2) les modifications doivent etre reproductibles, (3) le contexte du projet doit etre compris, et (4) les actions doivent etre reversibles. A partir de ces axiomes, on reconstruit l'architecture sans biais de conformite. Cela signifie, par exemple, que si un agent de code a besoin d'executer du code pour le valider, l'axiome de reproductibilite impose un environnement sandboxe et deterministic, pas simplement un terminal SSH.

2.2 Theorie des Contraintes (TOC)

Dans un systeme multi-agents, le goulot d'etranglement est presque toujours le LLM lui-meme : la latence d'inference et le cout par token dominant toutes les autres contraintes. L'optimisation doit donc se concentrer sur : (1) la reduction du nombre d'appels LLM par tache, (2) le choix du modele le moins cher qui repond au besoin de qualite, (3) le cache semantique pour eviter les appels redondants, et (4) le routing dynamique qui envoie les taches simples vers des modeles rapides et peu couteux. Toute optimisation ailleurs (base de donnees, reseau, CPU) est secondaire tant que le LLM reste le bottleneck.

2.3 Compression Architecturale

Applique aux agents IA, ce principe stipule que chaque composant doit etre justifie par une fonction critique. Si un module de memoire semantique est ajoute "au cas ou l'agent aurait besoin de se souvenir de conversations passees", mais qu'aucun cas d'usage actuel ne le necessite, il faut le supprimer (YAGNI). L'architecture MVP d'un agent doit commencer par le pipeline minimum : Input > Classification > LLM > Output, puis ajouter progressivement le RAG, la memoire, le routing dynamique et les outils selon les besoins reels observes.

2.4 Reversibilite Decisionnelle

Pour les agents IA, les decisions irreversibles (Type 1) incluent : le choix du framework d'orchestration (LangGraph vs Mastra vs custom), le schema de memoire persistante, et l'interface de communication entre agents. Ces decisions necessitent des semaines d'analyse, des POC et des RFC. Les decisions reversibles (Type 2) incluent : le prompt systeme, le modele LLM par default (changeable via router), la strategie de chunking, et le format de stockage des embeddings. Ces decisions doivent etre prises en moins d'une heure avec le principe "disagree and commit".

2.5 Pensee Systemique - Les Leviers

En appliquant la hierarchie des leviers de Donella Meadows aux agents IA, les interventions les plus puissantes (niveaux 1-3) sont : transcender le paradigme (passer d'un agent monolithique a un orchestration multi-agents), changer les buts du systeme (optimiser pour la satisfaction utilisateur plutot que pour la precision du LLM), et l'auto-organisation (permettre aux agents de choisir dynamiquement leurs outils et strategies). Les interventions les moins puissantes (niveaux 10-12) sont : ajuster les parametres de temperature, modifier les tailles de buffer, et changer les constantes de timeout. L'architecte d'elite intervient aux niveaux 1-3.

3. Patterns d'Orchestration Multi-Agents

Le document original identifie cinq patterns fondamentaux d'orchestration multi-agents. Chaque pattern a des forces et des faiblesses specifiques qui le rendent plus ou moins adapte a chaque type d'agent. Le choix du pattern d'orchestration est une decision Type 1 (irreversible) qui conditionne toute l'architecture subsequente.

Pattern	Topologie	Agent Dev	Agent SLIDES
Supervisor	Hub & Spoke	Secondaire	Principal
Pipeline	Sequentiel	Principal	Secondaire
Swarm	Peer-to-Peer	Expérimental	Non adapté
Hierarchique	CEO/Manager/Worker	Pour équipes larges	Utile à grande échelle
Debat	Pro vs Contra + Juge	Pour revue de code	Pour validation design

Tableau 1 : Mapping des patterns d'orchestration aux agents cibles

3.1 Agent Developpeur : Architecture Pipeline + Debat

L'Agent Developpeur fonctionne au mieux avec un pattern Pipeline pour le flux principal de generation de code, enrichi d'un pattern Debat pour la phase de validation. Le pipeline sequentiel garantit que chaque etape produit un artefact verifiable avant de passer a la suivante, ce qui est critique pour la fiabilite du code. Le schema d'orchestration est le suivant :

```
Input Utilisateur > Analyseur (comprehension requete) > Planificateur
(decomposition taches) > Generateur (code + tests) > Valideur (execution +
review) > Correcteur (si erreurs) > Livreur (commit + doc)
```

Chaque etape du pipeline est un agent specialise avec son propre prompt systeme, ses propres outils et sa propre logique de validation. Le Valideur utilise le pattern Debat : deux sous-agents (un "critique" et un "defenseur") evaluent le code genere, et un agent Juge arbitre les desaccords. Ce mecanisme double la qualite du code en interceptant 60-80% des erreurs que l'auto-validation d'un seul agent laisserait passer. La boucle Correcteur ne s'execute que si le Valideur detecte des problemes, et elle est limitee a 3 iterations pour eviter les boucles infinies.

3.2 Agent SLIDES : Architecture Supervisor

L'Agent SLIDES necessite un pattern Supervisor car la creation de presentations est un processus creatif qui requiert des allers-retours entre des sous-agents specialises differents. Un superviseur central coordonne les appels aux sous-agents et agregge leurs resultats en temps reel. Le schema :

```
Superviseur SLIDES > Agent Structure (plan/outline) > Agent Contenu (texte +
donnees) > Agent Design (layout + visuels) > Agent Validation (cohérence +
qualité) > Superviseur (agrégation)
```

Le Superviseur maintient un etat global de la presentation (nombre de slides, theme visuel, narrative arc) et decide dynamiquement quel sous-agent appeler ensuite. Par exemple, si l'Agent Contenu signale qu'une slide necessite un graphique de donnees, le Superviseur invoque l'Agent Design avec un contexte specifique pour creer la visualisation. Si l'Agent Validation detecte une incoherence entre le titre et le contenu d'une slide, le Superviseur renvoie la tache a l'Agent Contenu avec les corrections demandeess. Ce pattern est plus flexible que le pipeline sequentiel car il permet des retours en arriere et des

ajustements iteratifs.

4. Architecture RAG de Production

Le document original fournit une architecture RAG extremement detaillee (Section 7.1) qui constitue le coeur de la capacite de connaissance des deux agents. Sans RAG, un agent est limite a ses connaissances d'entrainement ; avec RAG, il accede en temps reel a la documentation, aux exemples de code, aux templates de design et aux best practices. L'architecture RAG de production comporte deux pipelines distincts : l'Ingestion et la Requete.

4.1 Pipeline d'Ingestion

Le pipeline d'ingestion transforme les sources brutes en vecteurs recherchables. Il comporte quatre etapes critiques. Premierement, le Loader charge les documents depuis des sources heterogenes (documentation web, depots Git, fichiers PDF, API). Deuxiemement, le Chunker decoupe les documents en segments semantiques et hierarchiques, pas simplement par nombre de tokens. Le chunking semantique preserve le contexte en respectant les frontieres naturelles des documents (paragaphes, fonctions, sections). Troisiemement, l'Enricher ajoute des metadonnees (source, date, type de contenu, resume generatif) et cree des index secondaires. Quatriemement, l'Embedder genere des vecteurs denses avec des modeles multiples (OpenAI ada-002 pour la qualite, BGE pour l'auto-hebergement) et des vecteurs creux (BM25/SPLADE) pour la recherche hybride.

Pour l'Agent Developpeur, les sources d'ingestion sont : la documentation des frameworks et langages, les snippets de code verifies, les schemas de base de donnees du projet, et l'historique des commits. Pour l'Agent SLIDES, les sources sont : les templates de presentation, les palettes de couleurs et typographies, les guides de design, et les exemples de slides par industrie. Le stockage utilise pgvector (si PostgreSQL est deja present) ou Qdrant (si plus de 10M vecteurs) avec un schema multi-vectoriel combinant vecteurs denses, vecteurs creux et metadonnees.

4.2 Pipeline de Requete

Le pipeline de requete est l'element qui distingue un RAG basique d'un RAG de production. Il comporte six etapes en cascade : (1) Analyse de la requete avec classification de l'intention (factual, creatif, code) et scoring de complexite ; (2) Expansion de la requete via HyDE (Hypothetical Document Embeddings) et generation de sous-questions ; (3) Recherche hybride combinant recherche dense (vectorielle), recherche creuse (BM25) et traversée de graphe de connaissances ; (4) Fusion par Reciprocal Rank Fusion (RRF) des resultats de chaque methode ; (5) Re-ranking via un cross-encoder (Cohere ou BGE-reranker) pour affiner la pertinence ; et (6) Compression du contexte et preparation des citations avant injection dans le LLM.

Le point critique est l'etape de re-ranking : sans elle, la precision du RAG chute de 40%. Le cross-encoder re-evalue chaque paire (requete, document) avec une comprehension profonde de la

semantique, capturant des nuances que la recherche vectorielle seule ne peut pas detecter. Pour l'Agent Developpeur, cela signifie que si l'utilisateur demande "comment configurer Prisma avec PostgreSQL", le re-ranker fera remonter la documentation specifique de Prisma plutot qu'un article generique sur PostgreSQL.

5. Routeur de Modeles Intelligent

Le document original (Section 7.3) decrit un routeur de modeles dynamique qui optimise le compromis cout/qualite/latence. C'est un composant essentiel pour les deux agents car il permet de reduire les couts de 60-90% sans degradation perceptible de la qualite. Le routeur fonctionne en trois couches : classification, decision et execution.

Complexite	Type de tache	Modele recommande	Cout approx.
Simple	Formatage, correction, résumé basique	DeepSeek / GPT-4o-mini	0.14\$/M tokens
Moyen	Generation de code standard, contenu de slide	Claude Haiku / GPT-4o-mini	0.15-1\$/M tokens
Complexe	Architecture, refactoring, design créatif	Claude Opus / GPT-4o	5-15\$/M tokens
Sensible	Données privées, code propriétaire	Ollama (local, privé)	Coût infra seule
Temps réel	Autocompletion, chat en direct	Groq (ultra-rapide)	0.59\$/M tokens

Tableau 2 : Stratégie de routing dynamique des modèles LLM

Le routeur implemente egalement trois strategies d'optimisation avancees. Premierement, la cascade : pour les taches de complexite incertaine, on essaie d'abord le modele le moins cher, et on escalade uniquement si le resultat est insuffisant (detecte par un classificateur de qualite). Deuxiemement, le cache semantique : si une requete est semantiquement equivalente ($similarite > 0.95$) a une requete precedente, on retourne la reponse cachee sans appeler le LLM. Troisiemement, le prompt caching : Anthropic et OpenAI offrent des reductions de 50-90% pour les prefixes de prompts reutilisés, ce qui est particulierement adapte aux prompts systemes des agents qui sont identiques d'un appel a l'autre.

6. Hierarchie de Memoire Persistante

Le document original (Section 7.4) definit une hierarchie de memoire en cinq couches qui est le coeur de la capacite d'apprentissage des agents. Sans memoire persistante, un agent est amnesique a chaque session. Avec la hierarchie complete, il accumule de l'experience, apprend des preferences utilisateur et s'amelior avec le temps.

Couch e	Type	Stockage	Capacité	Utilisation Agent
Working	Context window LLM	RAM (tokens)	~100K tokens	Tache en cours
Episodi c	Historique conversations	Redis	Sessions récentes	Contexte utilisateur
Semanti c	Faits utilisateur/domaine	pgvector	Illimitée	Préférences, connaissances
Procedu ral	Skills, how-to appris	KB + outils	Illimitée	Recettes, patterns
Archiv e	Passe compressé	S3 + index	Illimitée	Historique long terme

Tableau 3 : Hiérarchie de mémoire persistante pour agents IA

6.1 Techniques de Compression de Memoire

Le document identifie quatre techniques de compression essentielles pour gerer la memoire sur le long terme. La summarisation roulante : toutes les N messages, un LLM genere un resume de la conversation qui remplace les messages originaux dans la working memory, liberant de l'espace pour de nouvelles interactions. Le scoring d'importance : chaque element de memoire est evalue selon sa probabilite d'utilisation future, et seuls les elements au-dessus d'un seuil sont conserves en memoire rapide. La reflection : periodiquement, l'agent analyse ses interactions passees pour extraire des insights et des lecons apprises qui sont stockees dans la memoire semantique. Le decoupage par episodes : les conversations sont segmentees par sujet ou par periode temporelle, permettant de charger uniquement les episodes pertinents dans le contexte.

Pour l'Agent Developpeur, la memoire procedurale est la plus critique : elle stocke les patterns de code reussis, les corrections frequentes et les preferences de style. Pour l'Agent SLIDES, c'est la memoire semantique qui domine : preferences de design, palettes favorites, et styles de presentation par client ou par industrie. L'implementation utilise PostgreSQL + pgvector pour le stockage semantique, Redis pour la memoire episodique (avec TTL de 7 jours), et S3 pour l'archivage a long terme.

7. Boucle d'Auto-Amelioration

Le document original (Section 7.5) decrit une boucle d'auto-amelioration en cinq etapes qui est le mecanisme par lequel un agent s'amelior avec le temps. C'est ce qui distingue un agent basique d'un agent de classe mondiale. La boucle est : Execution > Observation > Reflection > Learning > Storage, puis Retrieval pour les taches futures.

Concretement, apres chaque tache executee, l'agent observe le resultat : la tache a-t-elle reussi ? Quels ont ete les points de friction ? L'utilisateur a-t-il du corriger le resultat ? Ces observations sont passees a un module de Reflexion (un LLM distinct ou le meme modele avec un prompt different) qui analyse ce qui a marche et ce qui a echoue. Le module de Learning extrait des patterns de succes et d'echec sous forme de "lecons apprises" qui sont vectorisees et stockees dans la memoire semantique. Lors de la prochaine tache similaire, le module de Retrieval charge automatiquement les lecons pertinentes dans le contexte du LLM.

Les techniques avancees mentionnees dans le document incluent : REFLEXION (Shinn et al.) qui utilise des reflexions linguistiques pour guider les actions futures, Self-RAG qui apprend a l'agent a evaluer sa propre generation, Chain-of-Verification (CoVe) qui genere des questions de verification pour detecter les hallucinations, et Tree of Thoughts (ToT) qui explore plusieurs chemins de raisonnement en parallele avant de selectionner le meilleur. Pour l'Agent Developpeur, la technique la plus efficace est CoVe : apres avoir genere du code, l'agent genere des questions de verification (ce code compile-t-il ? respecte-t-il les conventions du projet ? y a-t-il des vulnerabilites ?) et les resout pour valider sa propre production.

8. Securite IA en Profondeur

Le document original (Section 7.6) definit cinq couches de defense pour les systemes LLM. Ces couches sont absolument critiques pour les agents qui executent du code (Agent Developpeur) ou qui generent du contenu visible publiquement (Agent SLIDES). Un agent non securise est une faille de securite potentielle.

Couche	Mesures	Agent Dev	Agent SLIDES
1. Input	Injection detection, PII scrubbing, rate limiting	Critique	Important
2. Prompts	System prompt strict, format impose, few-shot defensifs	Critique	Critique
3. Execution	Whitelist outils, permissions, sandbox (Docker/Firecracker)	Vital	Modere
4. Output	Hallucination detection, PII leak, harmful content filter	Important	Critique
5. Observabilite	Logging, anomaly detection, cost monitoring	Important	Important

Tableau 4 : Couches de securite IA par agent

Pour l'Agent Developpeur, la couche Execution est vitale : tout code execute par l'agent doit l'etre dans un sandbox isole (Docker avec ressources limitees, pas d'accès reseau non autorise, timeout strict). La

couche Input est également critique car le code utilisateur peut contenir des injections de prompt malveillantes camouflées dans des commentaires ou des noms de variables. Pour l'Agent SLIDES, la couche Output est primordiale : le contenu genere ne doit pas contenir de données personnelles, de contenu nuisible ou de violations de droits d'auteur. Chaque sortie passe par un filtre de moderation avant d'etre presentee a l'utilisateur.

9. Architecture Cible : Agent Developpeur

Cette section synthetise les elements extraits du document original en une architecture concrete pour l'Agent Developpeur. L'architecture combine le pattern Pipeline (flux principal), le pattern Debat (validation), le RAG hybride (connaissance), le routeur de modeles (optimisation cout), la memoire en cinq couches (apprentissage), et la securite en profondeur (protection).

9.1 Vue d'Ensemble

L'Agent Developpeur est structure en six modules autonomes qui communiquent via un bus d'evenements asynchrone. Le module Analyseur reçoit la requete utilisateur et la decompose en sous-taches. Le module Planificateur cree un plan d'execution avec des dependances entre les sous-taches. Le module Generateur produit le code et les tests unitaires pour chaque sous-tache. Le module Valideur execute le code en sandbox et lance une revue de code (pattern Debat). Le module Correcteur applique les corrections necessaires (maximum 3 iterations). Enfin, le module Livreur commit le code, met a jour la documentation et notifie l'utilisateur.

Module	Responsabilité	Modèle LLM	Outils
Analyseur	Compréhension requête, décomposition	GPT-4o-mini	RAG, Parser
Planificateur	Plan d'exécution, dépendances	Claude Haiku	Graph builder
Générateur	Code + tests unitaires	Claude Opus / GPT-4o	File editor, RAG code
Valideur	Exécution sandbox, revue code	Claude Haiku (débat)	Docker, Linter, Test runner
Correcteur	Corrections basées sur erreurs	Claude Opus / GPT-4o	Diff, Patch
Livreur	Commit, doc, notification	GPT-4o-mini	Git, Doc generator

Tableau 5 : Architecture modulaire de l'Agent Développeur

9.2 Infrastructure Partagee

Les modules partagent une infrastructure commune composee de quatre composants. Le RAG Engine (pgvector + BM25 + re-ranker) fournit l'accès a la documentation technique, aux exemples de code et aux best practices. Le Model Router (classification + cascade + cache semantique) optimise le choix du

LLM pour chaque sous-tache. La Memoire Persistante (Redis episodique + pgvector semantique + S3 archive) permet l'apprentissage au fil du temps. Et le Bus d'Evenements (BullMQ + Redis) assure la communication asynchrone entre modules avec garantie de livraison.

L'ensemble est deploye sur une infrastructure Kubernetes avec auto-scaling base sur la longueur de la file d'attente des taches. Le modele de cout est base sur les credits IA du document original (Section 8.3) : chaque action consomme des credits proportionnels au cout reel du LLM, avec une marge de 20% pour couvrir les couts d'infrastructure. Le cache semantique reduit le nombre d'appels LLM de 40-60% en pratique, ce qui ameliore significativement le ratio cout/qualite.

10. Architecture Cible : Agent SLIDES

L'Agent SLIDES utilise le pattern Supervisor car la creation de presentations est un processus creatif qui necessite des ajustements iteratifs entre la structure, le contenu et le design. L'architecture comporte un Superviseur central et quatre sous-agents specialises, chacun avec son propre ensemble d'outils et de connaissances.

10.1 Vue d'Ensemble

Sous-Agent	Responsabilité	Modèle LLM	Connaissances RAG
Superviseur	Coordination, état global, routing	Claude Haiku	Templates de workflow
Structure	Plan, outline, narrative arc	Claude Opus	Storytelling, structures narratives
Contenu	Texte, données, citations	GPT-4o	Données sectorielles, citations
Design	Layout, visuels, couleurs, typographie	GPT-4o + Image Gen	Palettes, templates, guidelines
Validation	Cohérence, qualité, accessibilité	Claude Haiku	Standards, règles accessibilité

Tableau 6 : Architecture modulaire de l'Agent SLIDES

10.2 Flux de Creation d'une Presentation

Le flux de creation se deroule en quatre phases orchestrees par le Superviseur. Phase 1 - Structuration : l'utilisateur decrit sa presentation, l'Agent Structure cree un outline avec le nombre de slides, la narration et les points cles. Le Superviseur valide l'outline avec l'utilisateur. Phase 2 - Contenu : l'Agent Contenu genere le texte de chaque slide, les donnees chiffrées et les citations. Le Superviseur verifie la coherence narrative entre les slides. Phase 3 - Design : l'Agent Design cree le layout de chaque slide, selectionne les palettes de couleurs, genere les visuels et les graphiques. Le Superviseur s'assure de la coherence

visuelle a travers toutes les slides. Phase 4 - Validation : l'Agent Validation verifie la coherence texte/design, l'accessibilite (contraste, taille de police), et la qualite globale.

Chaque phase peut impliquer des allers-retours. Si l'Agent Validation detecte un probleme de design sur la slide 5, le Superviseur renvoie cette slide specifique a l'Agent Design avec les corrections demandees, sans regenerer l'ensemble de la presentation. Ce mecanisme de correction cible est beaucoup plus efficace et economique qu'une regeneration complete. De plus, le Superviseur maintient un etat global qui permet de garantir la coherence transversale : les couleurs de la slide 3 doivent correspondre a celles de la slide 7, le style de typographie doit etre uniforme, et la narrative doit suivre un arc logique.

10.3 Generation de Visuels

L'Agent Design utilise deux strategies pour les visuels. Pour les graphiques de donnees (barres, lignes, camemberts), il genere du code (matplotlib, ECharts ou D3.js) qui est execute cote serveur pour produire des images haute resolution. Pour les illustrations et images creatrices, il utilise un modele de generation d'images (DALL-E ou Stable Diffusion) avec des prompts specialises pour le style de presentation. Le prompt de generation inclut toujours le theme visuel de la presentation (palette, style) pour garantir la coherence.

11. Stack Technologique Recommandee

En appliquant le principe "Choose Boring Tech" du document original et la doctrine de selection database (PostgreSQL resout 95% des cas), voici la stack recommandee pour les deux agents. Cette stack privilegie la simplicite operationnelle, la maturite des outils et la coherence technologique (TypeScript de bout en bout).

Composant	Technologie	Raison
Framework API	Hono / Next.js API Routes	Edge-ready, TypeScript, léger
Orchestration Agents	LangGraph / Mastra	Stateful workflows, visualisation
Base de données	PostgreSQL + pgvector	RDB + vecteurs unifiés (95% des cas)
Cache / Queue	Redis + BullMQ	Cache, sessions, rate-limit, jobs
Workflows durables	Temporal.io	Sagas, compensation, reprise
LLM SDK	z-ai-web-dev-sdk	Multi-provider, routing intégré
Embeddings	OpenAI ada-002 / BGE	Qualité / auto-hébergement
Sandbox exécution	E2B / Firecracker	Isolation sécurisée pour code
Observabilité	OpenTelemetry + Grafana	Traces, métriques, logs unifiés
Déploiement	Vercel + Railway/Fly.io	MVP rapide, scaling progressif

11.1 Stack AI-Native (Levier 10x)

Le document original identifie une "Stack Fusion AI-Native" avec un effet de levier 10x. Pour les agents IA, cette stack se decline ainsi : Next.js 14 (App Router + RSC + Server Actions) pour le frontend, Hono pour les API edge, PostgreSQL + pgvector comme base de donnees unifiee (relationnel + vectoriel), Redis pour le cache et les queues, et le SDK multi-provider pour le routing dynamique des LLM. Le point cle est l'unification : un seul langage (TypeScript), une seule base de donnees (pgvector evite d'ajouter Qdrant/Pinecone), et un seul systeme de cache (Redis remplace BullMQ standalone, les sessions et le rate-limiting). Cette unification reduit la complexite operationnelle de 60% et accelere le developpement de 3x par rapport a une stack multi-systemes heterogene.

12. Anti-Patterns a Eviter

Le document original identifie dix anti-patterns majeurs. Sept d'entre eux sont directement applicables a la construction d'agents IA. Voici les anti-patterns specifiques a eviter absolument, avec leurs symptomes et corrections.

12.1 Microservices Prematures pour Agents

Symptomes : deployer l'Agent Developpeur et l'Agent SLIDES comme microservices separees alors qu'ils partagent 80% de l'infrastructure. Correction : commencer par un monolithe modulaire ou chaque agent est un module avec des bornes claires, puis extraire uniquement les composants qui necessitent un scaling independant (probablement le sandbox d'execution de code et le generateur d'images). La regle : ne passer aux microservices que si vous avez plus de 30 developpeurs ou des besoins de scaling heterogene avers.

12.2 Cache-First Aveugle

Symptomes : cacher toutes les reponses LLM sans reflechir a la fraicheur du contenu. Pour l'Agent Developpeur, cacher les snippets de documentation est legitime, mais cacher les resultats de generation de code est dangereux car le contexte du projet peut avoir change. Correction : cacher avec TTL court (5 minutes) par default, utiliser des tags de cache pour l'invalidation ciblee, et ne jamais cacher les resultats de generation creative ou les executions de code.

12.3 Observability Absente

Symptomes : production est une boite noire, pas de tracing des appels LLM, pas de monitoring des couts par utilisateur, pas d'alertes sur les taux d'erreur. C'est l'anti-pattern le plus courant dans les projets d'agents IA. Correction : implementer OpenTelemetry des le premier jour avec des traces pour chaque

appel LLM (modele, prompt, tokens, latence, cout), des metriques pour les taux de succes/echec par agent, et des alertes sur les anomalies de cout. Le monitoring des couts LLM est particulierement critique car une anomalie peut signifier une attaque par injection de prompt ou une boucle infinie.

12.4 Isolation Multi-Tenant Cassee

Symptomes : un utilisateur voit les donnees ou les conversations d'un autre utilisateur. C'est une catastrophe pour les agents IA qui manipulent du code propriétaire et des donnees sensibles. Correction : implementer Row Level Security (PostgreSQL RLS) au niveau de la base de donnees, utiliser AsyncLocalStorage pour propager le contexte tenant dans Node.js, et executer des tests d'isolation systematiques dans le pipeline CI/CD.

12.5 Sur-Ingenierie

Symptomes : ajouter un graphe de connaissances Neo4j alors que pgvector suffit, implementer CQRS + Event Sourcing pour un CRUD basique, deployer Kafka pour 100 evenements par jour. Correction : appliquer YAGNI absolu. Construire pour aujourd'hui plus 6 mois maximum. Refactoriser quand le besoin existe veritablement. PostgreSQL resout 95% des cas. Ne pas ajouter de systeme supplementaire sans une raison EXTREME.

13. Roadmap d'Implementation (12 Semaines)

Adapte du roadmap 24 mois du document original, ce plan accelere concentre les livrables essentiels sur 12 semaines en appliquant le principe de compression architecturale et le pattern "Startup Speed" (MVP en 2-4 semaines). Chaque phase produit un increment fonctionnel deployable.

Phase	Semaines	Livrables	Stack
Fondation	1-3	MVP Agent Dev (pipeline basique), API, Auth, DB	Next.js + Hono + PG + Redis
Intelligence	4-6	RAG hybride, Model Router, Cache sémantique	+ pgvector + Temporal
Mémoire	7-8	Mémoire 5 couches, Auto-amélioration loop	+ BullMQ workers
SLIDES	9-10	Agent SLIDES (supervisor + 4 sous-agents)	+ Image gen + ECharts
Production	11-12	Sécurité 5 couches, Observabilité, CI/CD	+ OTEL + Grafana + K8s

Tableau 8 : Roadmap d'implémentation sur 12 semaines

13.1 Phase Fondation (Semaines 1-3)

L'objectif est d'avoir un Agent Developpeur fonctionnel capable de lire une requete, generer du code et le retourner a l'utilisateur. Le pipeline est basique (Analyseur > Generateur > Livreur) sans validation ni correction automatique. L'infrastructure comprend : Next.js pour le frontend et les API routes, PostgreSQL pour les donnees utilisateur, Redis pour les sessions, et un seul modele LLM (GPT-4o-mini pour commencer). Le RAG est minimal : une collection pgvector avec les documentations les plus utilisees. Cette phase valide le produit avec de vrais utilisateurs et recueille les premiers retours.

13.2 Phase Intelligence (Semaines 4-6)

Cette phase enrichit l'Agent Developpeur avec les capacites qui font la difference entre un chatbot et un agent de production. Le RAG hybride (dense + sparse + re-ranking) est deploye avec un pipeline d'ingestion automatique des documentations. Le Model Router implemente le routing dynamique avec cascade et cache semantique. Les modules Valideur et Correcteur sont ajoutes au pipeline, avec le pattern Debat pour la revue de code. Temporal.io est integre pour les workflows durables (une generation de code peut prendre plusieurs minutes). Cette phase transforme l'agent d'un generateur passif en un systeme actif qui valide et corrige sa propre production.

13.3 Phase Memoire (Semaines 7-8)

La memoire en cinq couches est deployee : working memory (context window), episodique (Redis, TTL 7 jours), semantique (pgvector), procedurale (knowledge base + outils), et archive (S3). La boucle d'auto-amelioration est activee : apres chaque tache, l'agent observe, reflechit, extrait des lecons et les stocke pour les taches futures. Les BullMQ workers gerent les taches de fond (ingestion RAG, compression de memoire, generation de resume). Cette phase donne a l'agent la capacite d'apprendre de ses experiences et de s'ameliorer avec le temps.

13.4 Phase SLIDES (Semaines 9-10)

L'Agent SLIDES est construit sur l'infrastructure existante en ajoutant le pattern Supervisor avec ses quatre sous-agents. L'Agent Structure utilise Claude Opus pour creer des outlines narratifs de qualite. L'Agent Contenu genere le texte et les donnees avec GPT-4o. L'Agent Design cree les layouts et genere les visuels via un modele de generation d'images. L'Agent Validation verifie la coherence et l'accessibilite. Le Superviseur coordonne les appels et gere les retours en arriere. Cette phase etend la plateforme d'un agent a deux agents partageant la meme infrastructure.

13.5 Phase Production (Semaines 11-12)

La securite en cinq couches est deployee : validation d'input, prompts defensifs, sandbox d'execution (E2B/Firecracker pour l'Agent Dev), validation d'output (moderation de contenu pour l'Agent SLIDES), et observabilite complete. OpenTelemetry est integre pour les traces, metriques et logs. Grafana fournit les dashboards de monitoring. Le pipeline CI/CD est configure avec des tests d'isolation multi-tenant, des tests de securite (injection de prompt) et des tests de performance (latence P99 < 2s). Le deploiement

passer de Vercel/Railway à Kubernetes si le volume le justifie. Cette phase prépare la plateforme pour la mise en production avec de vrais utilisateurs payants.

14. Les 15 Commandements de l'Architecte d'Agents

Synthèse des 21 commandements du document original, filtrés et adaptés spécifiquement pour la construction d'agents IA. Ces principes sont les invariants qui guident chaque décision architecturale.

1. Optimisez pour l'INCERTITUDE, pas la prédiction. Les LLM sont non-déterministes ; votre architecture doit être résiliente aux réponses inattendues.
2. Choisissez BORING TECH pour les fondations. PostgreSQL + Redis + TypeScript résolvent 95% des cas sans ajouter de complexité.
3. Mesurez avant d'optimiser. Les LLM sont le goulot d'étranglement dans 80% des cas ; optimisez le routage et le cache avant tout.
4. Async par défaut, sync par exception. Les appels LLM sont lents ; chaque appel doit être non-bloquant.
5. Modular monolith d'abord, microservices au mérite. Ne séparez les agents que si le scaling le justifie réellement.
6. PostgreSQL + pgvector résout 95% des besoins de stockage. N'ajoutez Qdrant que si > 10M vecteurs.
7. Routeur de modèles obligatoire. Un seul modèle pour tout = gaspillage de 60-90% du budget IA.
8. Zero trust, défense en profondeur. Chaque input utilisateur est potentiellement une injection de prompt.
9. Observabilité non optionnelle. Sans traces des appels LLM, vous êtes aveugle sur les coûts et la qualité.
10. Cache sémantique = vaccin anti-coût. 40-60% des requêtes sont sémantiquement identiques.
11. Idempotence partout. Un appel LLM qui échoue doit pouvoir être réexécuté sans effet de bord.
12. Feature flags > déploiements coordonnés. Activez les nouveaux agents progressivement (1% > 10% > 100%).
13. Mémoire persistante = avantage compétitif. Un agent qui apprend est 10x plus précis qu'un agent amnésique.
14. First principles > best practices. Ne copiez pas l'architecture de Genspark ; décomposez vos besoins jusqu'aux axiomes.

15. Apprendre des post-mortems > célébrer les succès. Chaque erreur d'agent est une leçon à stocker dans la mémoire sémantique.